

Deep Learning, Introduction and Advanced Concepts

3. Generalization and Practical Techniques

Louis Bagot

January 8, 2026

Outline

1. Generalization and Regularization in Deep Learning
 - ▶ Definitions
 - ▶ Dropout
 - ▶ L2 regularization
2. Practical tricks and techniques in Deep Learning
 - ▶ Data augmentation
 - ▶ Transfer
 - ▶ Self-Supervision

Outline

1. Generalization and Regularization in Deep Learning

- ▶ Definitions
- ▶ Dropout
- ▶ L2 regularization

2. Practical tricks and techniques in Deep Learning

- ▶ Data augmentation
- ▶ Transfer
- ▶ Self-Supervision

Defining Generalization

Our true objective is generally not good performance on the training data, but to use the model on *data it hasn't seen before*:

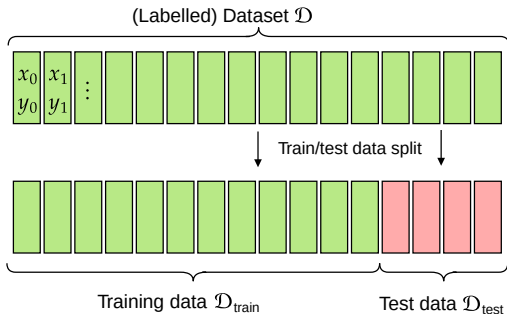
Generalization:

The ability to maintain good model performance beyond the training data.

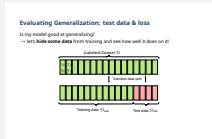
Evaluating Generalization: test data & loss

Is my model good at generalizing?

→ let's **hide some data** from training and see how well it does on it!

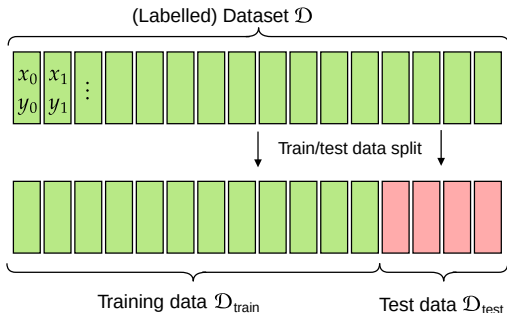


Notes for this slide



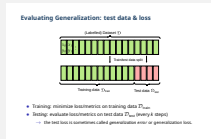
By "hiding from training" we mean the network / training algo cannot use the data to perform backpropagation and gradient descent. We will use the test data to *evaluate* training though, i.e., see how well the model is doing on data it hasn't seen.

Evaluating Generalization: test data & loss



- Training: minimize loss/metrics on training data $\mathcal{D}_{\text{train}}$
- Testing: evaluate loss/metrics on test data $\mathcal{D}_{\text{test}}$ (every k steps)
 - the test loss is sometimes called *generalization error* or generalization loss.

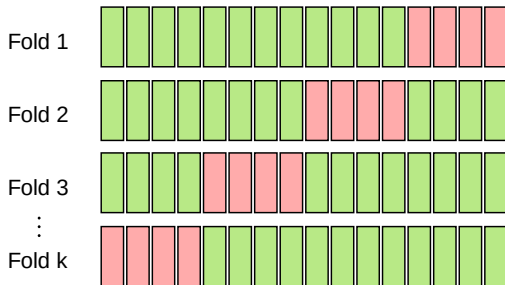
Notes for this slide



by “metrics” we mean all the things that we can use to determine if the model is doing good on some data. Loss is the main one, but we can think about accuracy/recall scores etc... and task-specific indications (e.g. total reward in Reinforcement Learning)

k -fold cross-validation

Don't have much data? Train & evaluate the model on many choices of test sets:



→ Compute average performance over k trainings

Hyperparameter tuning

I want to find the set of hyperparameters that generalizes best.

First idea:

Notes for this slide

Hyperparameter tuning

I want to find the set of hyperparameters that generalizes best.

First idea:

Reminder hyperparameters: set of values & choices that are fixed before training and not changed or learned. Contrasts with the parameters θ which the network learns.

Examples of hyperparameters from Course 1:

Number of hidden layers L

Size of hidden layers k

Activation function σ

Loss function $\mathcal{L}$

Learning rate η

Batch size n

Even the type of layers, choice of optimizer and its parameters, choice of initialization, normalizations...can be seen as hyperparameters.

Using the test dataset to fine-tune these hyperparameters is “cheating” in the sense that, by design, the test dataset is supposed to be “data we don’t have access to”.

Hyperparameter tuning

I want to find the set of hyperparameters that generalizes best.

First idea:

- try out hyperparameters and pick the ones that did best on the test dataset!

Hyperparameter tuning

I want to find the set of hyperparameters that generalizes best.

First idea:

try out hyperparameters and pick the ones that did best on the test dataset!

Problem:

Hyperparameter tuning

I want to find the set of hyperparameters that generalizes best.

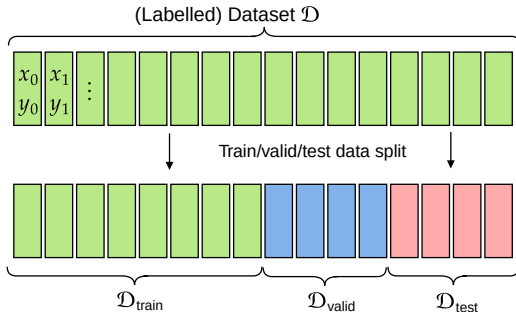
First idea:

try out hyperparameters and pick the ones that did best on the test dataset!

Problem:

it may be the hyperparameters that work best for *this specific* set of test data!

Validation set for hyperparameter search



Train on $\mathcal{D}_{\text{train}}$, optimize hyperparameters on $\mathcal{D}_{\text{valid}}$, evaluate on $\mathcal{D}_{\text{test}}$.

Regularization: reducing generalization error

Now we know what we need to do!

Regularization:

Any strategy to reduce test error, possibly at the expense of training error.

Generalization : what are the challenges?

Generalization:

The ability to maintain good model performance beyond the training data.

Generalization : what are the challenges?

Generalization:

The ability to maintain good model performance beyond the training data.

Main issues:

- *Overfitting*: the model learns the training data too closely, or by heart
- *Misaligned Data*: the training distribution is not representative of what we care about

Notes for this slide

Generalization : what are the challenges?

Generalization:

The ability to maintain good model performance beyond the training data.

Main issues:

- Overfitting: the model learns the training data too closely, or by heart
- Mismatched data: the training distribution is not representative of what we care about

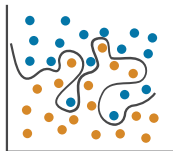
Note that we don't necessarily have the data to be able to evaluate the network on misaligned data. For example, if I have a dataset of animals in zoos, my model may be unable to identify animals in the wild due to different backgrounds and contexts. But it's a lot harder to get pictures of animals in the wild!

→ similarly to the train/test split, if we want to evaluate & solve this issue, we can recreate the data distribution problem manually by picking data. For example, picking only pictures of animals in cages for training, and evaluating if the model can identify animals in more open spaces but still in zoos.

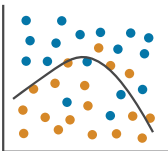
Overfitting and Underfitting intuition

Classification

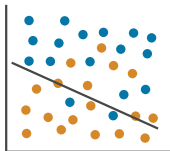
Overfitting



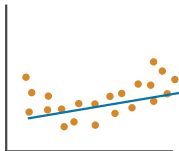
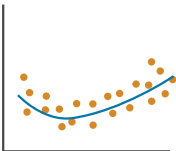
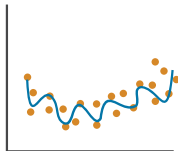
Right Fit



Underfitting



Regression



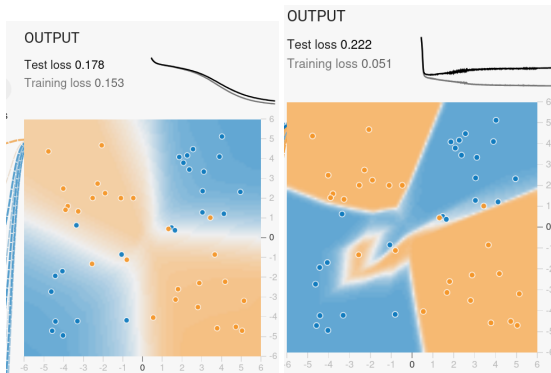
Notes for this slide

plot from [this blog post](#)



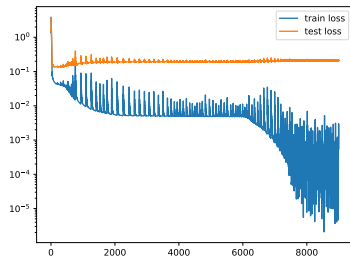
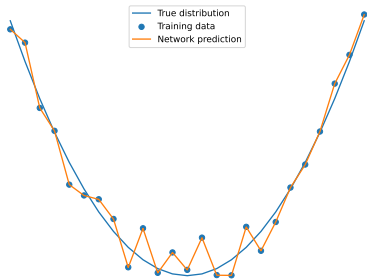
Overfitting concrete examples

The model starts learning even the noise in the data! (tensorflow playground)



Left: an early good prediction. Right: with more training the network learns noise

Minimal overfitting example with a neural network



Overfitting viewed from the loss and accuracy

In the vast majority of cases you cannot visualize your whole function output

→ **Visualize** train & test loss, accuracy! (and everything you can)

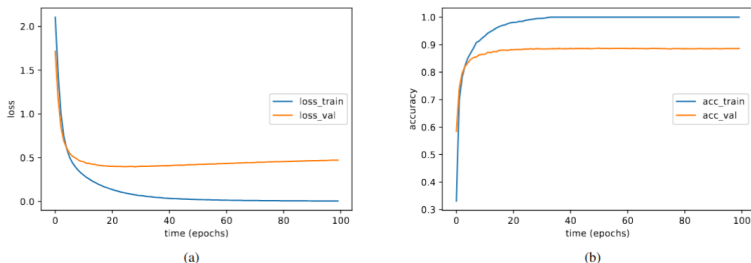


Fig. 2: Overfitting of the model on MNIST dataset. (a) shows that after a certain epoch, the training loss gets decreased while validation loss gets increased. (b) shows the gap between training and validation accuracy over the time.

Notes for this slide

Overfitting viewed from the loss and accuracy

In the vast majority of cases you cannot visualize your whole function output
→ **Visualize** train & test loss, accuracy! (and everything you can)

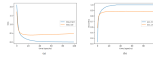


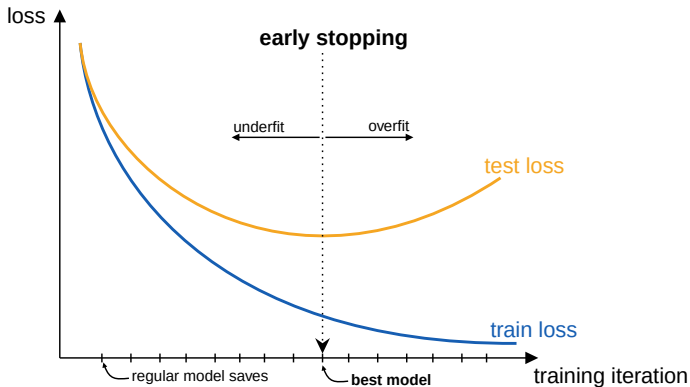
Fig. 2: Overfitting of the model on MNIST dataset. It shows that after a certain epoch, the training loss gets decreased while validation loss gets increased. It shows the gap between training and validation curves over the time.

plot from paper: [Overfitting Mechanism and Avoidance in Deep Neural Networks](#).

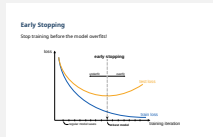
Plotting the losses (test and train) is critical in any Deep Learning scenario! In general, plot as many things as you can think of, to always check that you know what the model is doing.

Early Stopping

Stop training before the model overfits!



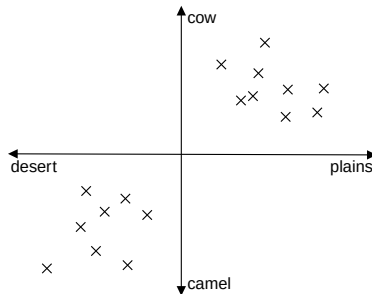
Notes for this slide



Since we can't know in advance when the model will start overfitting, we simply regularly save the weights of the model at different times θ^k (e.g. every 1000 gradient descent steps; value depends on application and model)

Addressing Overfitting: Lazy solutions

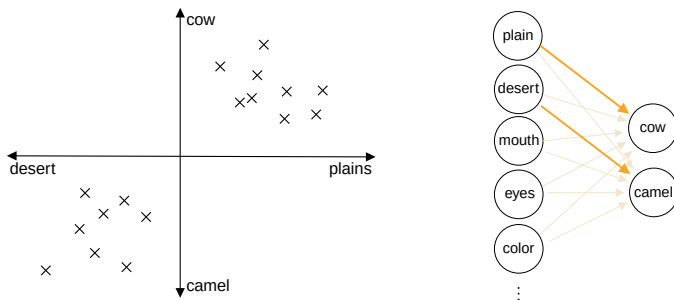
Imagine the network must distinguish cows from camels in pictures, with data distribution:



Problem:

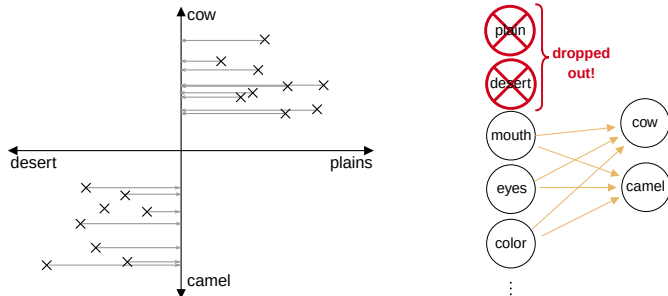
Addressing Overfitting: Lazy solutions

Imagine the network must distinguish cows from camels in pictures, with data distribution:



Problem: it's enough to identify the background to know the animal!
(much easier to do: average pixel value is either green or yellow)

Solution: Regularization with Dropout



Randomly turn off features (e.g. 50%) → the network must learn to do without!

Dropout against Overfitting

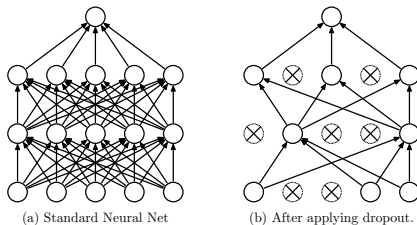
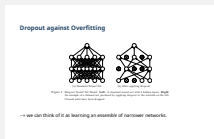


Figure 1: Dropout Neural Net Model. **Left:** A standard neural net with 2 hidden layers. **Right:** An example of a thinned net produced by applying dropout to the network on the left. Crossed units have been dropped.

→ we can think of it as learning an *ensemble* of narrower networks.

Notes for this slide



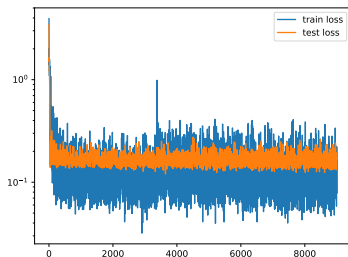
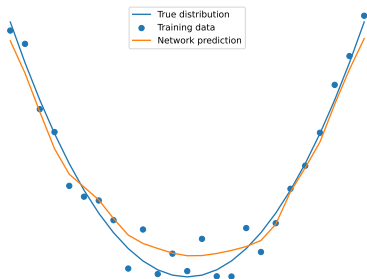
Plot from the [Dropout paper](#).

Implementation details: for each sample in the batch different neurons are (randomly) dropped out at all layers. So from the “ensemble model” perspective, each sample trains a different sub-model and the gradient descent computes an average over all the proposed weight updates.

We use a probability p that each neuron is dropped out, generally between 10 — 30%. In PyTorch this is achieved with `nn.Dropout(p)`, the technique is from 2012 but still actively used all the way to LLMs today.

During inference (when actually testing the network e.g. for computing the test metrics; also called “evaluation” in pytorch with `model.eval()`), Dropout is turned off, in other words, $p = 0$ temporarily.

Dropout ($p = 0.1$) on the minimal example



Regularization Techniques

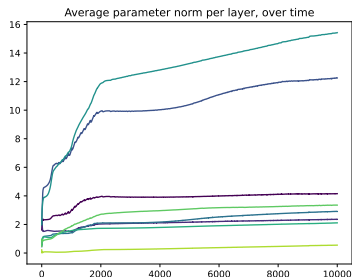
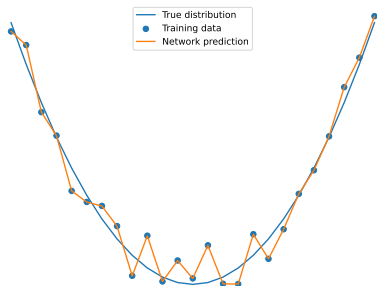
- Dropout
 - reduces overfit
 - easy to implement & no computational overhead
 - parameter p to fine-tune

Regularization Techniques

- Dropout
 - reduces overfit
 - easy to implement & no computational overhead
 - parameter p to fine-tune
- **L2 regularization: weight decay**

Intuition for weight decay

Limiting model capabilities: fitting noise tends to require *high weights* θ



→ intuition: **constrain weight size!**
i.e. force the net to use small θ_i

L2 Regularization: constraining weight size

Explicitly penalize big weights in the objective (loss) of the network:

$$\mathcal{L}_{\text{total}} = \mathcal{L} + \lambda \|\boldsymbol{\theta}\|_2^2$$

training loss balancing hyperparam. L2 norm of weights $\boldsymbol{\theta}$
(= $\sum_i \theta_i^2$)

Notes for this slide

L2 Regularization: constraining weight size

Explicitly penalize big weights in the objective (loss) of the network:

$$\mathcal{L}_{\text{total}} = \mathcal{L} + \lambda \sum_i \mathbf{w}_i^2$$

Annotations:
 - $\mathcal{L}$: Loss
 - λ : Regularization parameter
 - $\sum_i \mathbf{w}_i^2$: L2 norm of weights
 - $\mathbf{w}_i = \mathbf{w}_{i,j,k}$

AdamW is a variant of Adam that incorporates weight decay in a way that fits with the original Adam algo.

The new objective in the total loss equation is something we can optimize with gradient descent, since the norm (the square of features) is easily differentiable!

This idea of using a weighted sum of losses to combine objectives is very common in Deep Learning. We will use it to make a same network body do several things.

L2 Regularization: constraining weight size

Explicitly penalize big weights in the objective (loss) of the network:

$$\mathcal{L}_{\text{total}} = \mathcal{L} + \lambda \|\boldsymbol{\theta}\|_2^2$$

training loss balancing hyperparam. L2 norm of weights $\boldsymbol{\theta}$
($= \sum_i \theta_i^2$)

λ controls the balance:

- at $\lambda = 0$, the network simply solves its task (no changes)
- at $\lambda \rightarrow \infty$, the network only focuses on making each θ_i as small as possible
- in-between: balance the true objective with keeping weights small
 - ▶ Implementation in PyTorch: `AdamW(..., weight_decay=lambda)`

Notes for this slide

L2 Regularization: constraining weight size

Explicitly penalize big weights (in the objective (loss)) of the network:

$$\mathcal{L}_{\text{total}} = \mathcal{L} + \lambda \underbrace{\underbrace{\sum_i \theta_i^2}_{\text{L2 norm of weights}}}_{\text{L2 Regularization}} \quad (\lambda = \lambda_{\text{L2}})$$

λ controls the balance:

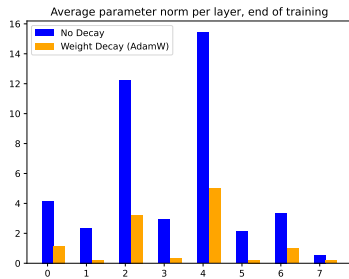
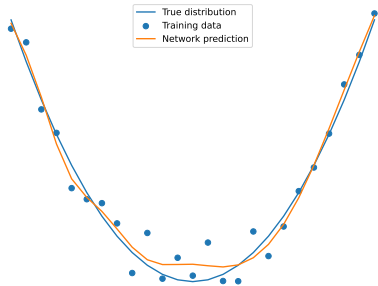
- at $\lambda \rightarrow 0$, the network simply solves its task (no changes)
- at $\lambda \rightarrow \infty$, the network only focuses on making each θ_i as small as possible
- in-between: balance the task objective with keeping weights small
 - Implementation in PyTorch: `AdamW(..., weight_decay=L2lambda)`

AdamW is a variant of Adam that incorporates weight decay in a way that fits with the original Adam algo.

The new objective in the total loss equation is something we can optimize with gradient descent, since the norm (the square of features) is easily differentiable!

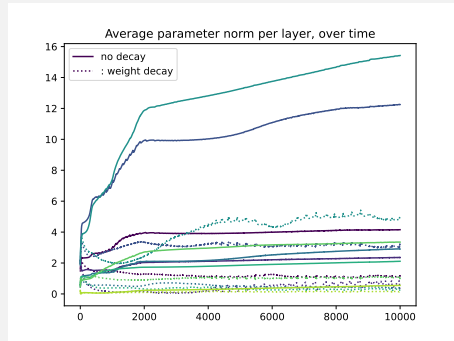
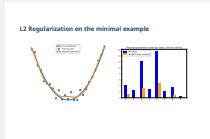
This idea of using a weighted sum of losses to combine objectives is very common in Deep Learning. We will use it to make a same network body do several things.

L2 Regularization on the minimal example



Notes for this slide

If you are interested, the norms over time:



Regularization Techniques

- Dropout
 - reduces overfit
 - easy to implement & no computational overhead
 - parameter p to fine-tune
- L2 regularization: weight decay
 - reduces overfit
 - easy to implement & no computational overhead
 - parameter λ to fine-tune

→ can generally use both + other methods

Outline

1. Generalization and Regularization in Deep Learning

- ▶ Definitions
- ▶ Dropout
- ▶ L2 regularization

2. **Practical tricks and techniques**

- ▶ Data augmentation
- ▶ Transfer
- ▶ Self-Supervision

The data problem

In 99% of cases, the main issue with using Deep Learning will be **data**

The data problem

In 99% of cases, the main issue with using Deep Learning will be **data**

- I don't have data
 - ▶ good luck!

The data problem

In 99% of cases, the main issue with using Deep Learning will be **data**

- I don't have data
- I have a small dataset

The data problem

In 99% of cases, the main issue with using Deep Learning will be **data**

- I don't have data
- I have a small dataset
 - ▶ my model is probably going to overfit
 - ▶ my model can't find relevant patterns

The data problem

In 99% of cases, the main issue with using Deep Learning will be **data**

- I don't have data
- I have a small dataset
- My data is not labelled (or only a small part)

The data problem

In 99% of cases, the main issue with using Deep Learning will be **data**

- I don't have data
- I have a small dataset
- My data is not labelled (or only a small part)
 - ▶ cannot work with Supervised Learning

The data problem

In 99% of cases, the main issue with using Deep Learning will be **data**

- I don't have data
- I have a small dataset
- My data is not labelled (or only a small part)
- My data is biased

The data problem

In 99% of cases, the main issue with using Deep Learning will be **data**

- I don't have data
- I have a small dataset
- My data is not labelled (or only a small part)
- My data is biased
 - ▶ some classes rarely show up (class imbalance)
 - ▶ datapoint distribution is not uniform
 - ▶ missing data

Using Expert knowledge

If you/someone has knowledge about the data or task, use it!

Using Expert knowledge

If you/someone has knowledge about the data or task, use it!

Expert knowledge:

prior knowledge about the task that allows to restrict the search space

Notes for this slide

Using Expert knowledge

If you/ someone has knowledge about the data or task, use it!

Expert knowledge:

prior knowledge about the task that allows to restrict the search space

Examples: (in order of the slide)

- values that are too high or too low (date > 2030, car weight < 1kg...), labelling mistakes (this is not a cat), corrupted data (image noise,)
- identify outliers (billionaires in salary data)
- turning classes into quantitative value or the opposite, splitting into interesting values...
- symmetries, constraints on the input/output which we can use...

Using Expert knowledge

If you/someone has knowledge about the data or task, use it!

Expert knowledge:

prior knowledge about the task that allows to restrict the search space

- identify mistakes in data
- remove useless features / datapoints
- normalize and pre-process data
- structure in the data (inputs or outputs), ...

Using Expert knowledge

If you/someone has knowledge about the data or task, use it!

Expert knowledge:

prior knowledge about the task that allows to restrict the search space

- identify mistakes in data
- remove useless features / datapoints
- normalize and pre-process data
- structure in the data (inputs or outputs), ...

→ *inductive biases* are an example of expert knowledge!

(I know that information in text/an image is local so i can use attention/convolution)

Data Augmentation to help with small datasets

Example of expert knowledge: **data augmentation**

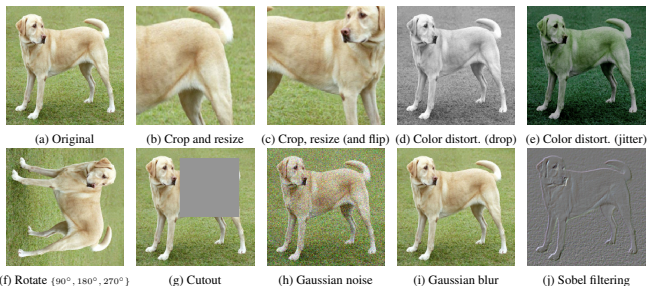


Figure 4. Illustrations of the studied data augmentation operators. Each augmentation can transform data stochastically with some internal parameters (e.g. rotation degree, noise level). Note that we *only* test these operators in ablation, the *augmentation policy* used to train our models only includes *random crop* (with *flip* and *resize*), *color distortion*, and *Gaussian blur*. (Original image cc-by: Von.grzanka)

→ I **know** that all those still correspond to the same image

→ LOTS more data!

Notes for this slide



Image from the [SimCLR paper](#).

Of course, if orientation / color are relevant to my specific application (clocks, flowers), then I don't include all of these.

Main idea: expert knowledge tells us how we can improve the data distribution by creating artificial data.

Notion of Latent Space

Notion of Latent Space

What does the network actually learn?

Notion of Latent Space

What does the network actually learn?

→ What do the *features* (neurons) at a given layer encode?

Notion of Latent Space

What does the network actually learn?

→ What do the *features* (neurons) at a given layer encode?

Intuition of learned latent space for language: (thousands of dimensions)



LLMs do this for entire sentences/prompts! → space of *meaning*

Notes for this slide

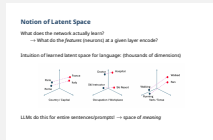


Image from [this blog post](#).

some methods explicitly try to obtain latent spaces nicely structured like this, most methods nowadays just see these additive patterns arise naturally, leading to the [linear representation hypothesis](#).

Notion of Latent Space

Sometimes called

- *Latent* space
- *Embedding* space
- *Feature* space / learned features
- *Representation* space / learned representation

Fine-tuning with Transfer Learning

Little data?

Use *existing models* as a strong starting point and **transfer** knowledge (features)

Notes for this slide

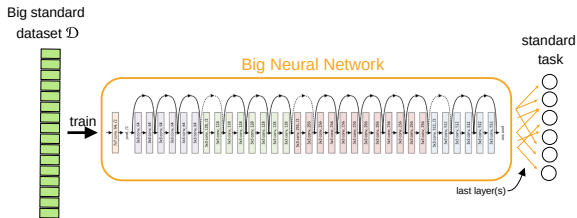
Fine-tuning with Transfer Learning

Little data?

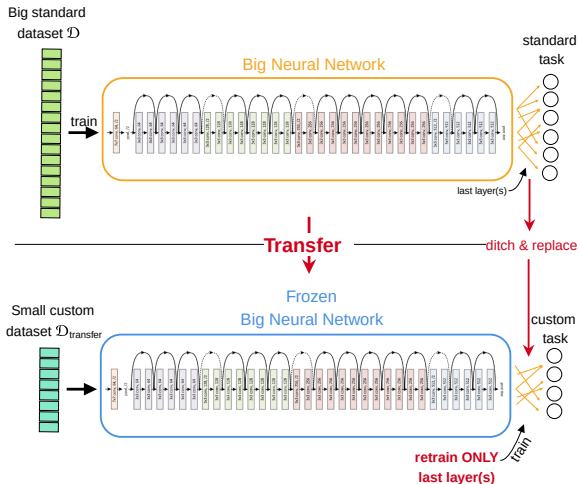
Use existing models as a strong starting point and **transfer** knowledge (features)

This is hugely common practice! Especially for tasks with big input spaces (e.g., vision, language) it is very rare to train your model from scratch. There are nowadays a huge amount of choices to start from for your model.

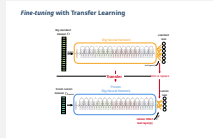
Fine-tuning with Transfer Learning



Fine-tuning with Transfer Learning



Notes for this slide



The core idea is that the big neural network will have learned *useful features about the input* (e.g., detecting objects for vision), so we can re-use those features in our little dataset if it requires the same types of features.

Example: you want to detect failures in pipes from pictures. You don't have many pictures. But the task is still fundamentally a vision task (pixels, colors, background, objects, patterns in the object) so using a big pre-trained neural network like a ResNet will be much more efficient than trying to solve the task from scratch.

The rise of Self-Supervision

Few/no labels, no prior info?

Learn useful features from *the input data alone*

The rise of Self-Supervision

Few/no labels, no prior info?

Learn useful features from *the input data alone*

Self-supervision:

train for generally useful *“pretext” tasks / pseudo-labels* (easy to generate)

Examples:

The rise of Self-Supervision

Self-supervision:

train for generally useful *“pretext” tasks / pseudo-labels* (easy to generate)

Examples:

- auto-encoder (input reconstruction)
- language:

The rise of Self-Supervision

Self-supervision:

train for generally useful *“pretext” tasks / pseudo-labels* (easy to generate)

Examples:

- auto-encoder (input reconstruction)
- language: similar word encoding, next token prediction, missing word prediction
- vision:

The rise of Self-Supervision

Self-supervision:

train for generally useful *“pretext” tasks / pseudo-labels* (easy to generate)

Examples:

- auto-encoder (input reconstruction)
 - language: similar word encoding, next token prediction, missing word prediction
 - vision: consistent prediction from augmented images
- hugely popular today!!

The rise of Self-Supervision

Self-supervision:

train for generally useful *“pretext” tasks / pseudo-labels* (easy to generate)

Examples:

- auto-encoder (input reconstruction)
 - language: similar word encoding, next token prediction, missing word prediction
 - vision: consistent prediction from augmented images
- hugely popular today!!
- perform **transfer** on the learned representation afterwards

Time for Presentations!

You know all you need to listen to the article presentations.